16/11/2025

DATA STRUCTURES:

A DATA STRUCTURE IS A WAY TO STORE DATA IN THE MEMORY OF A COMPUTER. IT IS IMPORTANT TO CHOOSE THE RIGHT DATA STRUCTURE, BECAUSE EACH OF THEM HAS ITS OWN ADVANTAGES AND DISADVANTAGES.

DYNAMIC ARRAYS:

A DYNAMIC ARRAY IS AN ARRAY WHOSE SIZE CAN BE CHANGED DURING EXECUTION OF THE PROGRAM. THE MOST POPULAR DYNAMIC ARRAY IN C++ IS THE VECTOR, WHICH CAN BE USED ALMOST LIKE AN ORDINARY ARRAY:

```
vector<int> v;
v.push_back(10);
v.push_back(20);
v.push_back(30);
// v now contains {10, 20, 30}
```

AFTER THIS WE CAN ACCESS ELEMENTS LIKE AN ORDINARY ARRAY:

```
cout << v[0] << "\n";
cout << v[1] << "\n";
cout << v[2] << "\n";
// prints 10, 20, 30
```

THE FUNCTION SIZE RETURNS THE NUMBER OF ELEMENTS IN THE VECTOR:

```
for (int i = 0; i < v.size(); ++i) {
    cout << v[i] << "\n";
}
```

ITERATES THROUGH THE VECTOR.

A SHORTER WAY:

```
for (auto x : v){
    cout << x << "\n";
}
```

THE FUNCTION BACK RETURNS THE LAST ELEMENT IN THE VECTOR, AND THE FUNCTION POP_BACK REMOVES THE LAST ELEMENT:

```
vector<int> v;
v.push_back(10);
v.push_back(20);
cout << v.back() << "\n"; // 20
v.pop_back();
cout << v.back() << "\n"; // 10
```

THE FOLLOWING CODE CREATES A VECTOR WITH FIVE ELEMENTS:

```
vector<int> v = {1, 2, 3, 4, 5};
```

ANOTHER WAY TO CREATE A VECTOR IS TO GIVE THE NUMBER OF ELEMENTS AND THE INITIAL VALUE FOR EACH ELEMENT:

```
vector<int> v(10); // Creates a vector of integers with 10 elements, all initialized to 0
vector<int> v2(5, 42); // Creates a vector of integers with 5 elements, all initialized to 42
```

PUSH_BACK() → O(1).

STRING → IS ALSO A DYNAMIC ARRAY, STRINGS CAN BE COMBINED USING THE + SYMBOL.

- SUBSTR (K, X) RETURNS A SUBSTRING AT POSITION K AND LENGTH X.
- FIND (t) FINDS THE POSITION OF THE FIRST OCCURANCE OF SUBSTRING t.

```
string a = "hatti";
string b = a+a;
cout << b << "\n"; // Output: hattihatti
b[5] = 'v';
cout << b << "\n"; // Output: hattivatti
string c = b.substr(3, 4);
cout << c << "\n"; // Output: tiva
```

SET STRUCTURES:

A SET IS A DATA STRUCTURE THAT MAINTAINS A COLLECTION OF ELEMENTS.

THE BASIC OPERATION OF A SET: INSERTION, SEARCH AND REMOVAL.

IN C++ WE HAVE 2 IMPLEMENTATIONS OF SET: SET AND UNORDERED_SET

- SET ↓ - SORTS - O(logn) - BINARY TREE
- UNORDERED_SET ↓ - O(1) - NO SORTING

THE FUNCTIONS OF SET ARE: INSERT (ADDS AN ELEMENT), COUNT (RETURNS THE NUMBER OF OCCURENCES OF AN ELEMENT) AND ERASE (REMOVES AN ELEMENT).

```
set<int> s;
s.insert(3);
s.insert(2);
s.insert(5);
cout << s.count(3) << "\n"; // 1
cout << s.count(4) << "\n"; // 0
s.erase(3);
s.insert(4);
cout << s.count(3) << "\n"; // 0
cout << s.count(4) << "\n"; // 1
```

A SET CAN BE USED MOSTLY LIKE A VECTOR, BUT IT IS NOT POSSIBLE TO ACCESS AN ELEM. USING [ ] NOTATION:

```
set<int> s = {1, 2, 3, 4, 5};
cour << s.size() << "\n"; // Output: 5
for (auto x : s) {
    cout << x << "\n";
}
```

AN IMPORTANT PROPERTY: SET DOES NOT HAVE DUPLICATES:

```
set<int> s;
s.insert(5);
s.insert(5);
s.insert(5);
cout << s.size(); // 1
```

C++ ALSO CONTAINS THE STRUCTURES MULTISET (SET) AND UNORDERED_MULTISET (UNORDERED_SET).

CONTAIN DUPLICATES

```
multiset<int> s;
s.insert(5);
s.insert(5);
s.insert(5);
cout << s.count(5); // 3
```

BUT THE FUNCTION ERASE REMOVES ALL ISTANCES OF AN ELEMENT:

```
s.erase(5);
cout << s.count(5); // 0
```

BUT YOU CAN REMOVE ONLY ONE INSTANCE:

```
s.erase(find(5));
cout << s.count(5); // 2
```

MAP STRUCTURES:

A MAP IS A ARRAY THAT CONSISTS OF KEY-VALUE-PAIRS.

THE KEY CAN BE ANY TYPE OF DATA. IN C++, WE HAVE MAP AND UNORDERED_MAP.

- MAP: - SORTS - $O(\log n)$ - BINARY TREE
- UNORDERED_MAP: - $O(1)$ - NO SORTING

```
map<string, int> m;
m["apple"] = 1;
m["banana"] = 2;
m["orange"] = 3;
cout << m["banana"] << "\n"; // 2
```

IF THE VALUE OF A KEY IS REQUESTED BUT THE MAP DOES NOT CONTAIN IT, THE KEY IS ADDED.

```
map<string, int> m;
cout << m["dawjda"] << "\n"; // 0
```

THE FUNCTION COUNT CHECKS IF A KEY EXISTS:

```
if(m.count("dawjdi")){
    // key exists
}
```

THE FOLLOWING CODE PRINTS ALL THE KEYS AND VALUES:

```
for(auto x : m){
    cout << m.first << " " << m.second << "\n";
}
```

ITERATORS AND RANGES:

AN ITERATOR IS A VARIABLE THAT POINTS TO AN ELEMENT IN A DATA STRUCTURE.

THE OFTEN USED: BEGIN AND END → RANGE OF ELEMENTS IN A DATA STRUCTURE.

BEGIN → FIRST ELEM.

END → AFTER THE LAST

$$S = \{3, 4, 6, 8, 10, 12, 14, 16, 18\}$$

S.BEGIN() → 3

S.END() → 18

WORKING WITH RANGES:

FOR EXAMPLE:

```
sort(s.begin(), s.end());
reverse(s.begin(), s.end());
random_shuffle(s.begin(), s.end());
```

→ SORTS A VECTOR.

→ REVERSES THE ORDER.

→ SHUFFLES THE ORDER RANDOMLY.

THESE FUNCTION CAN ALSO BE USED WITH ARRAYS, GIVING THE 2 POINTERS INSTEAD OF ITERATORS:

```
sort(s, s+n);
reverse(s, s+n);
random_shuffle(s, s+n);
```

→ n → SIZE OF THE ARRAY.

SET ITERATORS:

ITERATORS ARE OFTEN USED TO ACCESS ELEMENTS OF A SET.

```
set<int>::iterator it = s.begin();
```

→ POINTS TO THE SMALLEST ELEMENT IN A SET.

A SHORTER WAY:

```
auto it = s.begin();
```

WE CAN ACCESS TO THE ELEMENT THE ITERATOR POINTS USING * SYMBOL:

```
cout << *it << "\n";
```

WE CAN USE ITERATORS TO ITERATE THROUGH A SET:

```
for (auto it = s.begin(); it != s.end(); it++){
    cout << *it << "\n";
}
```

THE FOLLOWING CODE PRINTS THE LARGEST ELEMENT OF A SET:

```
auto it = s.end(); it--;
cout << *it << "\n";
```

THE FUNCTION FIND(x) RETURNS AN ITERATOR THAT POINTS TO THE ELEMEN WHOSE VALUE IS X, IF IT NOT FINDS IT THE ITERATOR WILL BE END().

```
auto it = s.find(x);
if(it == s.end()){
    // x is not found
}
```

OTHER STRUCTURES:

BITSET: IS AN ARRAY WHOSE VALUE IS EITHER 0 OR 1. FOR EXAMPLE:

```
bitset<10> s;
s[1] = 1;
s[3] = 1;
s[5] = 1;
s[7] = 1;
cout << s[3] << endl; // 1
cout << s[4] << endl; // 0
```

BENEFITS:

USE ONLY 1 BIT OF MEMORY.

A SHORTER WAY TO CREATE A BITSET:

```
bitset<10> s(string("1010101010"));  // from right to left
cout << s[4] << "\n"; // 0
cout << s[5] << "\n"; // 1
```

THE FUNCTION COUNT RETURNS THE NUMBER OF 1:

```
cout << s.count() << "\n"; // 5
cout << s.size() << "\n";  // 10
```

BIT OPERATION: AND (&), OR (|) AND XOR (^):

```
bitset<10> s(string("1010101010"));
bitset<10> t(string("1100110011"));
cout << (s & t) << endl;  // AND
cout << (s | t) << endl;  // OR
cout << (s ^ t) << endl;  // XOR
```

DEQUE: IS A DYNAMIC ARRAY, SLOWER THAN A VECTOR, IT HAS PUSH_BACK AND POP_BACK LIKE A VECTOR, BUT IT ALSO INCLUDES THE FUNCTIONS PUSH_FRONT AND POP_FRONT:

```
deque<int> d;
d.push_back(1); // [1]
d.push_back(2); // [1, 2]
d.push_front(3); // [3, 1, 2]
d.push_front(4); // [4, 3, 1, 2]
d.pop_back(); // [4, 3, 1]
d.pop_front(); // [3, 1]
```

PRORITY QUEUE:

A PRIORITY QUEUE MANTAINS A SET OF ELEMENTS. INSERTION AND REMOVAL TAKE $O(\log n)$ TIME AND RETRIEVAL TAKES $O(1)$ TIME. THE PRIORITY QUEUE IS FASTER THAN A SET, BECAUSE IT USES A CONTIGUOUS ARRAY TO STORE ELEMENTS (CACHE-FRIENDLY).